

11th Place Solution

Rank	11
Team	←→AB
Author	RihanPiggy
Topic ID	612186
Version	Original

Source: <https://www.kaggle.com/competitions/rsna-intracranial-aneurysm-detection/discussion/612186>

Retrieved with Kaggle CLI on 2026-07-07.

Congratulations to all the winners! Thanks to the organizers for hosting such an interesting competition. It is really a hard one and I enjoyed it a lot. Here let me share my solution.

TL;DR

My solution consists of 4 stages:

1. Generate brain mask with TotalSegmentator, train a 3D Unet to crop the ROI region which contains CoW Area.
2. Train 5 fold 2.5D CoW Seg model (cls loss + seg loss) and extract features.
3. Train 5 fold 2.5D Aneurysm model (cls + keypoint loss) and extract features.
4. Train RNNs on concatenated features from CoW Seg model and Aneurysm model across depth direction.

Preprocessing

I used [dicom2nifti](#) to convert dicom files to nifti files because TotalSegmentator uses this library.

I wrote a custom version to transform the coordinates and frame indexes from train_localizers.csv to the new ones which align with the nifti files.

Stage 1: Brain Mask

Brain Mask Generation

I used TotalSegmentator to generate brain masks for all scans.

Strangely, some of the CT scans have better brain masks generated by TotalSegmentator MRI than TotalSegmentator CT. Thus I generated 2 brain masks for each scan and selected the one with more pixels. I also filtered out small masks with less than 1 million pixels. Finally, I have 4386 brain masks.

3D Unet for Brain Segmentation

I trained a 3D Unet to segment the brain. The train code is reimplementation from [Qishen Ha's RSNA2022 solution](#).

I first pretrained the model with [CT](#) and [MRI](#) scans from Zenodo, which is a subset of the training data used to train TotalSegmentator. Then, I trained the model with 4386 brain masks. The model is trained with 5 folds. The input size is (128, 128, 128).

I used the trained 3D Unet to generate brain masks and crop the ROI region for all scans. The parameters for cropping are adjusted according to the CoWSeg and train_localizers.csv to ensure the cropped region contains the CoW Area.

Stage 2: CoW Segmentation Model

Unlike other competitors, I did not managed to train a good 3D CoW Segmentation model. Instead, I trained a 2.5D CoW Segmentation model with classification head. The purpose of this model is to extract positional features to feed into the final RNN model. According to my experience in RSNA2022, positional features boosted the CV. So although segmentation loss is even worse with 2.5D model (dice=0.56), the extracted features are still useful.

The training code is reimplement from [this amazing notebook](#) from UWMGI. The backbone I used is `tf_efficientnetv2_b0.in1k` (img size=224).

Also, using the OOF classification logits from the CoW Seg model like in stage 1, I filter out slices which do not contain CoW area.

Stage 3: Aneurysm Model

Similar to CoW Seg model, I trained a 2.5D Aneurysm model with classification head and keypoint head to extract features.

The key for the successful training of this model is as follows:

- 3 rounds of negative sampling. In the first round, I used classification logits from CoW Seg model to sample slices which contains vessels. In the next 2 rounds, I used OOF classification logits from the Aneurysm model itself to sample hard negative slices.
- Heuristic cropping to make the model focus on the vessel area. Crop ratio is adjusted according to the coordinates. (ensuring all the coordinates are in the cropped region)
- Focal loss with $\alpha=0.75$
- Self distillation with OOF logits to deal with label noise.
- EMA

The backbones I used in this stage are:

- `coatnet_rmlp_2_rw_384`: 5 fold, `img_size=384`
- `coatnet_rmlp_2_rw_384`: all data train. `img_size=384`
- `tf_efficientnetv2_s.in21k_ft_in1k`: another 5 fold, `img_size=384`

All data train used a different revision of dataset with less hard negative images. I first trained a 5 fold model and recorded the epoch with best CV to calculate the total training steps for all data train.

Stage 4: RNN Model

I trained 4 types of RNN models for each aneurysm model. The model starts to overfit quickly, EMA and heavy mixup(mixup ratio=0.5) are used to regularize the training.

For 5 fold Coatnet:

- Residual LSTM: aneurysm feature + CoW seg feature
- Residual GRU: aneurysm feature only
- BiLSTM: aneurysm feature only
- Bert: aneurysm feature only

For 5 fold Efficientnet:

- Residual LSTM: aneurysm feature + CoW seg feature
- Residual GRU: aneurysm feature + CoW seg feature
- LSTM: aneurysm feature only
- Bert: aneurysm feature only

For all data train Coatnet:

- Residual LSTM: aneurysm feature + CoW seg feature
- Residual GRU: aneurysm feature + CoW seg feature
- BiLSTM: aneurysm feature + CoW seg feature
- Bert: aneurysm feature + CoW seg feature

The ensemble of these RNNs reached auc=0.8909 and by self-distillation with OOF logits, another submission reached auc=0.9035

- submission 1: CV: 0.8909, Public: 0.87689, Private: 0.82201
- submission 2: CV: 0.9035, Public: 0.86703, Private: 0.81321

I am still very confused about the huge gap between Public and Private scores.

Maximizing GPU Utilization

To maximize the inference speed, I used Monai to preprocess the images on GPU. I used T4 x 2 and split the patient volume into 2 parts to parallelize the inference of 2.5D models. For RNN models, I simply split the models into 2 groups and run them in parallel on 2 GPUs. That's how I managed to finish the inference of 11 aneurysm models and 45 RNN models within 12 hours.

I suffered from timeout issue in the final days. Hoping that Kaggle will fix the new evaluation framework.

Supplemental Q&A

Tom · 2025-10-17T12:42:27.913000

@honglihang that's a good solution, especially utilizing 2.5d segmentation, I would like to use it in my next rsna competition. But seems one of your stage causes overfit, I think improving its generalization can make this solution to 1st.

RihanPiggy · 2025-10-17T13:25:15.393000

Thank you. I kept the kfold same in all the stages, so I believe there is no leak in training process, and public LB is quite correlated to CV, which is very confusing for me.

Considering that test scans are collected from different sites, maybe there is a domain shift in test data which cause the brain segmentator not working well? Another possibility is that there is an issue in dicom tag because I used tags to perform orienting and scaling.